

Perguntas frequentes

Análise Léxica e Sintática

Conteúdo autoral derivado do lesson model aprovado; a bibliografia é complementar.

Pergunta: Qual é o papel de Análise do código fonte no fluxo de análise do código?

Resposta: Etapa utilizada nas implementações de linguagens de programação por compilação, interpretação pura ou sistemas híbridos. No escopo da aula, ela envolve a análise léxica e a análise sintática.

Fontes: presentation, slide 3, paraphrase

Pergunta: Como reconhecer FNB na descrição da sintaxe em um exemplo?

Resposta: A FNB fornece uma descrição clara e concisa da sintaxe e pode servir de base para o analisador sintático, facilitando sua manutenção.

Fontes: presentation, slide 5, paraphrase

Pergunta: Que erro conceitual deve ser evitado ao explicar Analisador léxico e analisador sintático?

Resposta: A análise de um processador de linguagens é dividida em analisador léxico, associado a autômato finito e gramática regular, e analisador sintático ou parser, associado a autômato com pilha e GLC ou FNB.

Fontes: presentation, slide 4, paraphrase

Pergunta: Com que outra etapa ou conceito Separação da análise léxica e da análise sintática se relaciona?

Resposta: A separação favorece a simplicidade, pois permite técnicas menos complexas no analisador léxico; a eficiência, por possibilitar sua otimização; e a portabilidade, por isolar partes lexicais dependentes da plataforma.

Fontes: presentation, slide 6, paraphrase

Pergunta: Como verificar se Análise Léxica foi aplicado corretamente?

Resposta: Casamento de padrões que identifica subcadeias no código fonte, agrupa-as logicamente e fornece ao parser códigos internos correspondentes. O parser chama o analisador léxico quando precisa do próximo token.

Fontes: presentation, slide 7, paraphrase; presentation, slide 9, paraphrase

Pergunta: Qual é o papel de Lexema e token no fluxo de análise do código?

Resposta: Lexema é a subcadeia reconhecida no código fonte. Token é o código interno atribuído à categoria do lexema; por exemplo, diferentes nomes de variáveis podem receber IDENT, enquanto o símbolo de soma pode receber OP_SOMA.

Fontes: presentation, slide 7, paraphrase

Pergunta: Como reconhecer Abordagens para construir o analisador léxico em um exemplo?

Resposta: O analisador léxico pode ser obtido por uma ferramenta a partir da descrição formal dos tokens, implementado como programa baseado em um diagrama de estados ou construído por meio de uma tabela elaborada manualmente a partir desse diagrama.

Fontes: presentation, slide 9, paraphrase

Pergunta: Que erro conceitual deve ser evitado ao explicar Classes de caracteres?

Resposta: Agrupamentos como letras e dígitos substituem transições individuais para cada caractere, reduzindo a extensão e a complexidade do diagrama de estados usado no reconhecimento de identificadores e inteiros.

Fontes: presentation, slide 10, paraphrase; presentation, slide 11, paraphrase

Pergunta: Com que outra etapa ou conceito Reconhecimento de palavras reservadas se relaciona?

Resposta: Palavras reservadas e identificadores podem ser reconhecidos inicialmente pelo mesmo padrão. Depois, uma tabela determina se o lexema reconhecido como possível identificador corresponde a uma palavra reservada.

Fontes: presentation, slide 12, paraphrase

Pergunta: Como verificar se Funções getChar, addChar e lookup foi aplicado corretamente?

Resposta: getChar obtém o próximo caractere e determina sua classe; addChar incorpora o caractere ao lexema em processamento; lookup consulta a cadeia formada para verificar, entre outras possibilidades, se ela é uma palavra reservada e retornar seu código.

Fontes: presentation, slide 15, paraphrase

Pergunta: Qual é o papel de Objetivos da análise sintática no fluxo de análise do código?

Resposta: O parser deve localizar erros sintáticos, apresentar diagnósticos e recuperar-se rapidamente, além de produzir a árvore de análise sintática ou ao menos um caminho correspondente nessa árvore.

Fontes: presentation, slide 18, paraphrase

Pergunta: Como reconhecer Análise Sintática Descendente em um exemplo?

Resposta: Abordagem top down que constrói a árvore a partir da raiz, emprega derivação mais à esquerda e seleciona a regra do não-terminal corrente com base nos primeiros tokens que ele pode produzir.

Fontes: presentation, slide 19, paraphrase; presentation, slide 20, paraphrase

Pergunta: Que erro conceitual deve ser evitado ao explicar Análise Sintática Ascendente?

Resposta: Abordagem bottom up que começa pelas folhas e busca reduzir uma subcadeia correspondente ao lado direito de uma regra, reconstruindo no sentido inverso uma derivação mais à direita. A família LR é apresentada como a principal família de algoritmos ascendentes.

Fontes: presentation, slide 19, paraphrase; presentation, slide 22, paraphrase

Pergunta: Com que outra etapa ou conceito Deslocamento e redução se relaciona?

Resposta: Na análise sintática ascendente, deslocar move o próximo símbolo da entrada para a pilha; reduzir substitui uma subcadeia reconhecida pelo lado esquerdo da regra correspondente. Essas são as ações centrais dos analisadores shift-reduce da família LR.

Fontes: presentation, slide 22, paraphrase; sebesta-11ed, p. 202, paraphrase; sebesta-11ed, p. 204, paraphrase

Pergunta: Como verificar se Complexidade da análise sintática foi aplicado corretamente?

Resposta: Algoritmos aplicáveis a todas as gramáticas não ambíguas podem apresentar complexidade $O(n^3)$. Compiladores restringem-se a subconjuntos dessas gramáticas para realizar a análise em tempo linear $O(n)$, em que n é o tamanho da entrada.

Fontes: presentation, slide 24, paraphrase

Pergunta: Qual é o papel de Análise Sintática Descendente Recursiva no fluxo de análise do código?

Resposta: Implementação formada por uma coleção de subprogramas, geralmente com um subprograma para cada não-terminal. Os subprogramas percorrem as regras e produzem a análise em ordem descendente.

Fontes: presentation, slide 26, paraphrase; sebesta-11ed, p. 208, paraphrase

Pergunta: Como reconhecer FNBE para expressões simples em um exemplo?

Resposta: A FNBE apresentada organiza expressões em *expr*, *term* e *factor*. *expr* trata soma e subtração entre termos; *term* trata multiplicação e divisão entre fatores; *factor* reconhece identificador, constante inteira ou expressão entre parênteses.

Fontes: presentation, slide 27, paraphrase

Pergunta: Que erro conceitual deve ser evitado ao explicar Processamento do lado direito de uma regra?

Resposta: Para cada terminal no lado direito, o parser compara o símbolo com `nextToken` e sinaliza erro se não houver casamento. Para cada não-terminal, chama o subprograma correspondente. Quando um terminal casa, o analisador léxico fornece o próximo token.

Fontes: presentation, slide 28, paraphrase; sebesta-11ed, p. 192, paraphrase

Pergunta: Com que outra etapa ou conceito Convenção de `nextToken` se relaciona?

Resposta: Cada rotina do analisador sintático deixa em `nextToken` o próximo token ainda não processado. Assim, a rotina seguinte pode examinar esse token para continuar a análise ou escolher uma regra.

Fontes: presentation, slide 30, paraphrase; sebesta-11ed, p. 192, paraphrase

Pergunta: Como verificar se Lookahead foi aplicado corretamente?

Resposta: Quando um não-terminal possui mais de uma regra, o próximo token da entrada é comparado com os primeiros tokens que cada alternativa pode gerar. Uma alternativa compatível é escolhida; a ausência de casamento caracteriza erro sintático.

Fontes: presentation, slide 31, paraphrase; sebesta-11ed, p. 189, paraphrase

Pergunta: Qual é o papel de Recursão à esquerda no fluxo de análise do código?

Resposta: Uma gramática com recursão à esquerda direta ou indireta não pode ser usada diretamente por um analisador sintático descendente. A transformação apresentada separa alternativas recursivas $A\alpha$ das alternativas não recursivas β e introduz um novo não-terminal A' .

Fontes: presentation, slide 36, paraphrase

Pergunta: Como reconhecer FIRST em um exemplo?

Resposta: $FIRST(\alpha)$ é o conjunto de símbolos terminais que podem aparecer no início de uma cadeia derivada de α . Se α puder derivar a cadeia vazia, ϵ também pertence a $FIRST(\alpha)$.

Fontes: presentation, slide 38, paraphrase

Pergunta: Que erro conceitual deve ser evitado ao explicar Teste de Disjunção par a par?

Resposta: Para cada par de alternativas $A \rightarrow \alpha_i$ e $A \rightarrow \alpha_j$, os conjuntos $FIRST(\alpha_i)$ e $FIRST(\alpha_j)$ devem ter intersecção vazia. Se houver intersecção, o lookahead pode não determinar de forma única qual alternativa deve ser usada.

Fontes: presentation, slide 38, paraphrase; presentation, slide 39, paraphrase

As formulações são autorais e não reproduzem exercícios da bibliografia.